

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 6

Student Name: M. Vanisree

Roll No.: A24126552125

Date: 25-08-2026

TITLE

Implement Modules in NodeJS

AIM

To understand and implement different **built-in and user-defined modules in Node.js**, including the File System, Path, OS, HTTP, Events, URL, and Crypto modules, and to create and use a custom mathematical module.

OBJECTIVE

The objectives of this experiment are:

- To understand the concept of modules in Node.js.
 - To use the built-in **File System (fs) module**.
 - To perform file creation, reading, appending, and deletion operations.
 - To use the **Path (path) module** for handling file paths.
 - To retrieve system information using the **OS (os) module**.
 - To understand the basic usage of the **HTTP module**.
 - To create and handle custom events using the **Events module**.
 - To parse URLs using the **URL module**.
 - To generate hashes using the **Crypto module**.
 - To create and use a **user-defined module**.
 - To connect a Node.js application with **MongoDB using Mongoose**.
-

THEORY

Node.js provides a **module system** that allows developers to divide an application into smaller and reusable components. Modules can contain functions, variables, objects, or classes that can be imported and used in other JavaScript files.

Node.js provides several **built-in modules** that can be directly imported using the `require()` function.

The major modules used in this experiment are:

- **File System (fs)** – Used to create, read, modify, and delete files.
- **Path (path)** – Used to work with file and directory paths.
- **OS (os)** – Provides information about the operating system and system resources.
- **HTTP (http)** – Used to create HTTP servers.
- **Events (events)** – Used to create and handle custom events.
- **URL (url)** – Used to parse and manipulate URLs.
- **Crypto (crypto)** – Provides cryptographic functionality such as hashing.
- **User-defined module** – Allows programmers to create reusable functions in separate files.

The `module.exports` object is used to export functions or values from a custom module, while `require()` is used to import them.

The experiment also demonstrates connecting a Node.js application to **MongoDB** using the **Mongoose**

library.

ALGORITHM / PROCEDURE

1. Install Node.js on the system.
 2. Create a new Node.js project folder.
 3. Initialize the project using npm init.
 4. Create a JavaScript file for implementing Node.js modules.
 5. Import the required built-in modules using require().
 6. Use the fs module to create and read a file.
 7. Append additional information to the file.
 8. Use the path module to obtain file name, extension, directory, and joined path.
 9. Use the os module to display operating system and memory information.
 10. Create an HTTP server using the http module.
 11. Create and emit a custom event using the events module.
 12. Use the URL class to extract URL information.
 13. Generate a SHA-256 hash using the crypto module.
 14. Create a separate math.js file containing mathematical functions.
 15. Export the functions using module.exports.
 16. Import the custom module into the main JavaScript file.
 17. Execute the addition and subtraction functions.
 18. Install and import Express and Mongoose.
 19. Connect the application to MongoDB using Mongoose.
 20. Run the program and verify the output.
-

PROGRAM

A. File System, Path, OS, Events, URL and Crypto Modules

app.js

```
// to write file
const fs = require("fs");
console.log("Program started");
fs.writeFile("data.txt", "Hello Node.js", (err) => {
  if(err){
    console.log(err);
    return;
  }
  console.log("File created successfully");
});

//to read the file
fs.readFile("data.txt", "utf8", (err, data) => {
  if(err){
    console.log(err);
    return;
  }
  console.log(data);
});

// to add information
fs.appendFile("data.txt", "\nWelcome to Node.js", (err) => {
  if (err) {
    console.log(err);
    return;
  }
  console.log("Data added");
});

//delete the file
// fs.unlink("data.txt", (err) => {
//   if(err){
//     console.log(err);
//     return;
//   }
//   console.log("File deleted");
// })
```

```
// PATH MODULE
const path = require("path");
const filePath = "c:\\Users\\anits-csm\\Documents\\a24126552125\\Weeks\\app.js";
console.log("File name : ", path.basename(filePath));
console.log("Extension : ", path.extname(filePath));
console.log("Directory : ", path.dirname(filePath));
console.log("Joined path: ", path.join("folder", "data", "file.txt"));

//OS MODULE
const os = require("os");
console.log("Operating Systems:", os.platform());
console.log("Operating Systems:", os.arch());
console.log("Operating Systems:", os.cpus());
console.log("Operating Systems:", os.freemem());
console.log("Operating Systems:", os.totalmem());
console.log("Operating Systems:", os.homedir());

//HTTP MODULE
// const http = require("http");

// const server = http.createServer((req, res) => {
//   res.write("Hello from Node.js!");
//   res.end();
// });

// server.listen(3000, () => {
//   console.log("Server running on port 3000");
// });

//EVENTS MODULE
const EventEmitter = require("events");
const myEvent = new EventEmitter();
myEvent.on("login", () => {
  console.log("user logged in");
});
myEvent.emit("login");

//URL MODULE
const myUrl = new URL(
  "https://example.com/products?id=10&name=phone"
);
console.log("Protocol:", myUrl.protocol);
```

```

console.log("Protocol:", myUrl.protocol);

console.log("Host:", myUrl.hostname);

console.log("Path:", myUrl.pathname);

console.log("Query:", myUrl.search);

//CRYPTO MODULE
const crypto = require("crypto");

const hash = crypto
  .createHash("sha256")
  .update("hello")
  .digest("hex");

console.log(hash);

```

B. User-Defined Module

math.js

```

function add(a, b){
  return a + b;
}

function sub(a,b){
  return a - b;
}

module.exports = {
  add, sub
};

```

```

const math = require("./math");
console.log(math.add(10,20));
console.log(math.sub(20,10));

```

C. Express and MongoDB Connection

server.js

```

const express = require("express");
const mongoose = require("mongoose");

const app = express();

mongoose.connect("mongodb://127.0.0.1:27017/testDB")
  .then(() => {
    console.log("MongoDB connected");
  })
  .catch((err) => {
    console.log(err);
  });

app.listen(3000, () => {
  console.log("Server running");
});

```

CODE EXPLANATION

Code / Function	Explanation
-----------------	-------------

<code>require("fs")</code>	Imports the File System module.
<code>fs.writeFile()</code>	Creates a file and writes data into it.
<code>fs.readFile()</code>	Reads the contents of a file.
<code>fs.appendFile()</code>	Adds additional data to an existing file.
<code>fs.unlink()</code>	Deletes a file.
<code>require("path")</code>	Imports the Path module.
<code>path.basename()</code>	Returns the file name from a path.
<code>path.extname()</code>	Returns the extension of a file.
<code>path.dirname()</code>	Returns the directory portion of a path.
<code>path.join()</code>	Joins multiple path segments into a single path.
<code>require("os")</code>	Imports the OS module.
<code>os.platform()</code>	Returns the operating system platform.
<code>os.arch()</code>	Returns the system CPU architecture.
<code>os.cpus()</code>	Returns information about the available CPU cores.
<code>os.freemem()</code>	Returns the amount of free system memory.
<code>os.totalmem()</code>	Returns the total system memory.
<code>os.homedir()</code>	Returns the user's home directory.
<code>require("http")</code>	Imports the HTTP module.
<code>http.createServer()</code>	Creates an HTTP server.
<code>server.listen()</code>	Starts the HTTP server on a specified port.
<code>require("events")</code>	Imports the Events module.
<code>EventEmitter</code>	Used to create and handle custom events.
<code>myEvent.on()</code>	Registers an event listener.
<code>myEvent.emit()</code>	Triggers an event.
<code>new URL()</code>	Creates a URL object for parsing URL information.
<code>myUrl.protocol</code>	Returns the protocol of the URL.
<code>myUrl.hostname</code>	Returns the host name of the URL.
<code>myUrl.pathname</code>	Returns the path portion of the URL.
<code>myUrl.search</code>	Returns the query string of the URL.
<code>require("crypto")</code>	Imports the Crypto module.
<code>createHash("sha256")</code>	Creates a SHA-256 hashing object.

<code>.update()</code>	Provides data to the hash function.
<code>.digest("hex")</code>	Generates the hash value in hexadecimal format.
<code>function add()</code>	Defines an addition function in the custom module.
<code>function sub()</code>	Defines a subtraction function in the custom module.
<code>module.exports</code>	Exports functions from the custom module.
<code>require("./math")</code>	Imports the user-defined math.js module.
<code>mongoose.connect()</code>	Connects the Node.js application to MongoDB.
<code>app.listen()</code>	Starts the Express server.

TEST CASES AND EXECUTION DATA

Test Case	Component Tested	Action / Input	Expected Result	Actual Result	Status
TC-01	File System Module	Execute <code>fs.writeFile()</code> and <code>fs.readFile()</code>	File should be created and its contents should be displayed	File created and contents displayed successfully	Pass
TC-02	File System Module	Execute <code>fs.appendFile()</code>	New information should be added to data.txt	Data appended successfully	Pass
TC-03	Path Module	Execute <code>basename()</code> , <code>dirname()</code> , <code>dirname()</code> and <code>join()</code>	File path information should be displayed	File name, extension, directory and joined path displayed correctly	Pass
TC-04	OS Module	Execute OS information functions	System information should be displayed	Platform, architecture, CPU and memory information displayed	Pass
TC-05	Events Module	Emit the login event	"User logged in" should be displayed	"User logged in" displayed successfully	Pass
TC-06	URL Module	Parse the given URL	Protocol, host, path and query should be displayed	URL details displayed correctly	Pass
TC-07	Crypto Module	Generate SHA-256 hash for "hello"	Hash value should be generated	SHA-256 hash generated successfully	Pass
TC-08	User-Defined Module	Call <code>add(10,20)</code> and <code>sub(20,10)</code>	Addition should return 30 and subtraction should return 10	Correct results displayed	Pass
TC-09	MongoDB Connection	Start the Node.js application with MongoDB running	Application should connect to testDB	"MongoDB connected" displayed	Pass

OUTPUT

Output 1: File System Module

The program creates data.txt with the following content:

```
@mVanisree →.../Full-stack-lab/Record/Week-6/src (main) $ node builtinModules.js
Program started
File name : C:\Users\anits-csm\Documents\a24126552125\Week5\app.js
Extension : .js
Directory : .
Joined path: folder/data/file.txt
Operating Systems: linux
Operating Systems: x64
Operating Systems: [
  {
    model: 'AMD EPYC 7763 64-Core Processor',
    speed: 0,
    times: { user: 145200, nice: 1740, sys: 94510, idle: 213340, irq: 0 }
  },
  {
    model: 'AMD EPYC 7763 64-Core Processor',
    speed: 0,
    times: { user: 147000, nice: 1320, sys: 94640, idle: 218360, irq: 0 }
  }
]
Operating Systems: 6079082496
Operating Systems: 8330444800
Operating Systems: /home/codespace
user logged in
Protocol: https:
Host: example.com
Path: /products
Query: ?id=10&name=phone
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
Data added
File Craeted successfully
Hello Node.js
```

Output 2: User-Defined Module

```
@mVanisree →.../Full-stack-lab/Record/Week-6/src (main) $ node local.js
30
10
```

RESULT

Thus, the different built-in and user-defined modules in Node.js were

successfully implemented. File operations were performed using the File System module, path and system information were obtained using the Path and OS modules, events and URLs were handled using the Events and URL modules, and hashing was performed using the Crypto module. A custom mathematical module was also created and successfully imported and executed. Finally, a Node.js application was successfully connected to MongoDB using Mongoose.